



Gestion de la localisation défi 3

Projet : MOBITER

Documentation technique interne

Version 1.0

Yoan Douarre

Sherpa Engineering

31 juillet 2026

Table des matières

1	Comparaison de Kitware SLAM avec DLIO	4
1.1	Fonctionnement du Kitware SLAM	4
1.1.1	Prétraitement et extraction des caractéristiques géométriques	4
1.1.2	Estimation de l'odométrie LiDAR	4
1.1.3	Construction de la carte et optimisation	5
1.1.4	Ajouts spécifiques du Kitware SLAM	5
1.2	Fonctionnement du SLAM DLIO	5
1.2.1	Prétraitement et conditionnement des données denses	6
1.2.2	Correction de mouvement en temps continu et génération de l'a priori	6
1.2.3	Recalage géométrique direct <i>Scan-to-Map</i> (GICP)	6
1.2.4	Fusion et mise à jour d'état par observateur géométrique non linéaire	7
1.2.5	Synthèse des apports de DLIO par rapport aux approches classiques	7
1.3	Résultats expérimentaux	8
2	Modifications apportées à DLIO	11
3	Modifications de la gestion de la localisation	14
3.1	Brique de localisation	14
3.1.1	Intégration de DLIO	14
3.1.2	Ajout d'un modèle de covariance pour le SLAM	15
3.2	Brique de mapping	15
3.2.1	Copie locale de la carte avant le saut de localisation	15
3.2.2	Effacement des mappings avec une localisation incertaine	15

Table des figures

1	Comparaison du Kitware SLAM avec DLIO sur une trajectoire de 320 mètres en environnement géométriquement riche	8
2	Environnement d'essais de l'INRAE Aubière	8
3	Comparaisons du Kitware SLAM avec DLIO sur une trajectoire de 900 mètres en environnement géométriquement pauvre	9
4	Environnement d'essais de l'INRAE Montoldre	9
5	Taux de dérive de DLIO en environnement géométriquement pauvre	10
6	Mapping avec la version originale de DLIO	11
7	Position en Z de DLIO estimée avec l'IMU du Ouster	12
8	Position en Z de DLIO estimée avec l'IMU de la Xsens	12
9	Comparaison des mappings avant et après modification de DLIO	13
10	Arbre des transformations TF utilisé pour la localisation	14

Introduction

Pour fonctionner correctement, la stack MOBITER nécessite une localisation **la plus précise possible** et **continue**. Ces conditions étaient assurées lors du défi 1, durant lequel une localisation GNSS RTK était disponible en permanence. Cependant, à partir du défi 2, des pertes temporaires du signal GNSS ont commencé à être simulées sur des courtes périodes (<2 min).

Afin de permettre à la stack de continuer à fonctionner malgré ces interruptions, une solution de localisation basée sur un SLAM LiDAR a été intégrée. Le SLAM LiDAR de Kitware était alors utilisé, avec une extrapolation de sa pose à partir des données IMU afin d'obtenir une estimation à plus haute fréquence. Une brique de gestion de localisation a également été développée afin d'assurer la commutation entre les différentes sources de localisation lors des coupures du GNSS RTK, permettant ainsi de maintenir une estimation de pose précise et continue pendant ces périodes.

Cette solution s'est montrée suffisante pour le second défi, mais présentait néanmoins des limitations importantes pour la suite.

La principale limitation provenait de la dérive intrinsèque du SLAM, dont l'amplitude dépend fortement de la géométrie et de la richesse de l'environnement observé. En plus de fausser la trajectoire du robot, cette dérive pouvait progressivement compromettre la continuité de la localisation, notamment lors du retour du GNSS RTK après une période prolongée en SLAM. Afin d'éviter une discontinuité brutale de la pose estimée, le noeud de localisation évaluait alors la dérive accumulée par le SLAM au moment de la reconnexion GNSS et empêchait la commutation si celle-ci entraînait un saut de localisation trop important. Dans ce cas, le robot poursuivait son déplacement en utilisant uniquement l'estimation SLAM jusqu'à la fin du scénario. De plus, la brique du SLAM se retrouvait être le processus le plus lourd de toute la stack, ce qui finirait par poser des problèmes de performance au fur et à mesure de l'évolution de cette dernière.

Pour le troisième et dernier défi, les périodes de perte GNSS pouvant être significativement plus longues, nous avons donc décidé de remplacer le SLAM LiDAR de Kitware utilisé par le SLAM DLIO (Direct LiDAR-Inertial Odometry), une solution présentant une dérive plus faible, et d'ajouter une gestion de la discontinuité de la localisation pour que le robot puisse suivre au mieux son scénario lors de la coupure GNSS mais également pouvoir réutiliser l'information du GNSS RTK lors de la transition SLAM→GNSS, et ce peu importe la dérive.

1 Comparaison de Kitware SLAM avec DLIO

La principale limitation du Kitware SLAM est que ce dernier estime une localisation uniquement à partir des données LiDAR. Cette approche rend l'estimation fortement dépendante de la géométrie et de la richesse de l'environnement observé, mais également sensible aux distorsions du scan LiDAR dues notamment aux mouvements du véhicule lors de l'acquisition.

Afin de réduire ces limitations, une nouvelle approche basée sur le SLAM DLIO (*Direct Lidar-Inertial Odometry*) a été étudiée. Contrairement à un LiDAR SLAM classique, DLIO exploite conjointement les informations LiDAR et IMU afin d'obtenir une estimation de mouvement plus robuste et plus précise.

1.1 Fonctionnement du Kitware SLAM

Le Kitware SLAM est basé sur l'algorithme LOAM (*LiDAR Odometry And Mapping in Real-time*)[1] qui se décompose en trois grandes étapes :

1. Prétraitement et extraction des caractéristiques géométriques du scan LiDAR
2. Estimation de l'odométrie LiDAR
3. Construction de la carte et optimisation

1.1.1 Prétraitement et extraction des caractéristiques géométriques

La première étape consiste à prétraiter le nuage de points LiDAR afin de réduire les effets de distorsion liés au mouvement du véhicule pendant l'acquisition, puis à extraire les informations géométriques nécessaires au recalage. Contrairement à une approche utilisant l'ensemble des points du scan, LOAM sélectionne uniquement les points présentant une information géométrique suffisamment pertinente afin de limiter le coût de calcul tout en conservant les contraintes nécessaires à l'estimation du mouvement.

Deux types de caractéristiques sont principalement utilisés :

- Les points d'arêtes (*edge features*), correspondant aux zones présentant une forte variation de courbure dans le nuage de points. Ces points permettent de contraindre les mouvements autour des structures linéaires telles que les angles de bâtiments ou les contours d'obstacles.
- Les points plans (*surface features*), correspondant aux zones de faible courbure. Ils représentent généralement des surfaces régulières telles que le sol, les murs ou les grandes surfaces planes.

Ces caractéristiques sont ensuite utilisées lors des étapes de recalage afin d'estimer la transformation entre deux acquisitions LiDAR.

1.1.2 Estimation de l'odométrie LiDAR

La seconde étape consiste à estimer le déplacement relatif du robot entre deux scans successifs. Pour cela, LOAM réalise un recalage de type *scan-to-scan* entre les caractéristiques extraites du scan courant et celles du scan précédent.

L'estimation de la transformation est obtenue en minimisant deux types d'erreurs géométriques :

- Une distance point-droite pour les caractéristiques d'arêtes.
- Une distance point-plan pour les caractéristiques de surface.

L'optimisation de ces contraintes permet d'obtenir la transformation rigide entre deux positions successives du LiDAR. Cette étape fournit une estimation de mouvement à une fréquence supérieure à celle de la reconstruction de carte, mais reste sensible aux erreurs de recalage qui peuvent progressivement s'accumuler sous forme de dérive.

1.1.3 Construction de la carte et optimisation

La troisième étape consiste à construire progressivement une carte de l'environnement et à corriger la dérive accumulée lors de l'odométrie LiDAR.

Contrairement à l'étape précédente, qui réalise un recalage entre deux scans successifs, cette étape réalise un recalage du scan courant avec une carte locale (*scan-to-map*). Les caractéristiques extraites du nouveau scan sont alors alignées avec les éléments déjà présents dans la carte afin de raffiner l'estimation de pose.

Cette optimisation permet d'obtenir une trajectoire plus cohérente sur le long terme et une carte plus précise.

1.1.4 Ajouts spécifiques du Kitware SLAM

Bien qu'il soit difficile de trouver de la documentation sur le fonctionnement détaillé du Kitware SLAM, nous savons que ce dernier intègre une optimisation de graphe de poses[2], permettant de réaliser une correction de l'ensemble du mapping lorsqu'une fermeture de boucle est détectée, ainsi qu'une détection de features plus riche avec notamment la possibilité de détecter des points de rupture d'intensité et des blobs de points.

De plus, il permet l'injection de données provenant d'autres capteurs[3] tels qu'une IMU, un GNSS, une INS, une odométrie des roues, une caméra RGB pour la détection de tags, etc.

Cependant les essais réalisés n'ont pas permis d'observer de gain de performance, en particulier pour l'injection de données IMU, d'où le choix de n'utiliser que la version purement LiDAR et d'intégrer une IMU séparément pour obtenir une localisation à plus haute fréquence. Certaines features du SLAM ont également dû être désactivées ou limitées pour pouvoir maintenir un fonctionnement en temps réel dans la stack.

1.2 Fonctionnement du SLAM DLIO

DLIO (*Direct Lidar-Inertial Odometry*)[4] est un algorithme d'odométrie LiDAR-Inertielle hautement efficient et léger, conçu pour maintenir une localisation précise et construire une carte 3D dense même lors de mouvements agressifs ou dans des environnements peu structurés.

Contrairement aux approches classiques (telles que LOAM ou Kitware SLAM) qui reposent sur l'extraction de primitives géométriques (arêtes et plans) et sur un recalage intermédiaire *scan-to-scan*, DLIO adopte une architecture condensée : il travaille directement sur le nuage de points dense et réalise un recalage direct *scan-to-map* grâce aux garanties de convergence formelles de son observateur d'état.

L'architecture de DLIO s'articule autour de quatre étapes principales :

1. Prétraitement et conditionnement des données denses
2. Correction de mouvement en temps continu et génération de l'a priori
3. Recalage géométrique direct *Scan-to-Map* (GICP)
4. Fusion et mise à jour d'état par observateur géométrique non linéaire

1.2.1 Prétraitement et conditionnement des données denses

La première étape consiste à aligner temporellement et spatialement les acquisitions du LiDAR (10–20 Hz) et de l'IMU à haute fréquence (100–500 Hz) dans le repère du robot $\mathcal{R}$ situé à son centre de gravité.

Afin d'éviter toute perte d'information géométrique critique, DLIO rejette la détection de primitives (arêtes, plans, gradients d'intensité, etc.) ou le sous-échantillonnage agressif par grille voxel. Le nuage de points brut est conservé dans sa quasi-totalité, à l'exception d'un simple filtrage spatial de proximité destiné à éliminer les points appartenant à la structure propre du robot et la possibilité d'ajouter un léger sous-échantillonnage (0.05 à 0.1m) pour les LiDARs très denses.

1.2.2 Correction de mouvement en temps continu et génération de l'a priori

Pendant la durée du balayage LiDAR, les mouvements du robot induisent des distorsions spatiales importantes. Pour éliminer ces déformations sans recourir aux simplifications classiques (vitesse constante) ni au coût de calcul élevé d'autres méthodes d'interpolation, DLIO introduit une approche séquentielle de type *coarse-to-fine* :

1. **Coarse step** : Les mesures IMU comprises entre deux scans successifs sont intégrées numériquement sous l'hypothèse d'un modèle de mouvement d'ordre supérieur à secousse (*jerk*) linéaire et accélération angulaire constantes. Cela permet d'obtenir une estimation de l'état du robot à chaque nouvelle valeur IMU.
2. **Fine step** : Pour chaque point p_k^n du nuage acquis à l'instant relatif t_n , DLIO se sert des deux états estimés par l'IMU encadrant t_n pour résoudre un jeu d'équations analytiques sous forme close paramétré uniquement par l'horodatage t_n . Cette formulation fermée permet de calculer de manière exacte et entièrement parallélisable la transformation spécifique de dé-distorsion (*deskewing*) pour chaque point.

Ce traitement projette directement l'ensemble des points corrigés dans le repère monde $\mathcal{W}$, fournissant simultanément une estimation initiale $T_{prior}^{\mathcal{W}}$ plus précise pour l'étape de recalage géométrique.

1.2.3 Recalage géométrique direct *Scan-to-Map* (GICP)

Grâce à la qualité de l'estimation initiale produite lors de la correction de mouvement, DLIO supprime l'étape intermédiaire de recalage *scan-to-scan* habituellement nécessaire et effectue directement un alignement *scan-to-map*.

Le nuage corrigé et transformé $\hat{\mathcal{P}}_k^{\mathcal{W}}$ est aligné avec une sous-carte locale $\hat{\mathcal{S}}_k^{\mathcal{W}}$, construite à partir de keyframes.

L'alignement repose sur l'algorithme GICP (*Generalized Iterative Closest Point*)[5], une méthode qui modélise la surface locale autour de chaque point par une matrice de covariance, puis

réalise une correspondance de surface plan-à-plan en cherchant à minimiser la distance de Mahalanobis entre les points.

La résolution de ce problème d'optimisation non linéaire fournit une transformation correctrice $\Delta\hat{T}_k$, permettant d'obtenir la pose affinée du robot $\hat{T}_k^{\mathcal{W}}$.

1.2.4 Fusion et mise à jour d'état par observateur géométrique non linéaire

Pour fusionner les données LiDAR et IMU, DLIO intègre un *observateur géométrique non linéaire hiérarchique*.

Cet observateur prend en entrée la pose affinée issue du recalage GICP et la combine aux mesures IMU brutes à haute fréquence pour estimer l'état complet du système $\hat{X}_k = [p^{\mathcal{W}}, q^{\mathcal{W}}, v^{\mathcal{W}}, b^a, b^{\omega}]^{\top}$ (position, orientation, vitesse, et biais des accéléromètres et gyromètres).

Basé sur la théorie de la contraction, **cet observateur garantit une convergence globale théoriquement prouvée**.

Les états estimés sont utilisés pour les prochaines intégrations IMU, fermant ainsi la boucle d'estimation.

1.2.5 Synthèse des apports de DLIO par rapport aux approches classiques

L'architecture de DLIO apporte plusieurs bénéfices majeurs :

- **Couplage serré et convergence garantie** : L'intégration étroite (couplage serré) entre les contraintes géométriques du LiDAR et la dynamique inertielle au sein d'un observateur contractile prévient la divergence de l'état, même lors de manœuvres agressives.
- **Correction de distorsion continue et parallélisable** : L'utilisation d'équations analytiques d'ordre supérieur (secousse/accélération angulaire) élimine les artefacts de déformation sans l'impact algorithmique d'une optimisation spline lourde.
- **Efficacité computationnelle** : En supprimant la détection de primitives, le recalage *scan-to-scan* et en utilisant *Fast GICP*[6] pour grandement diminuer le temps de calcul de l'alignement GICP, DLIO réduit la charge de calcul de près de 20 % par rapport à l'état de l'art (tel que FAST-LIO2[7] ou LIO-SAM[8]) tout en améliorant la précision de la trajectoire.

1.3 Résultats expérimentaux

Malgré les limitations des fonctionnalités du SLAM de Kitware pour assurer son fonctionnement en temps réel, **on observe une réduction d'environ 50% d'utilisation de RAM et de charge CPU avec le SLAM DLIO par rapport à notre version allégée de Kitware**. De plus, ces limitations et l'ajout d'une IMU en couplage serré dans le processus de SLAM font que l'on obtient une localisation plus précise avec DLIO.

Voici un exemple de comparaison de qualité de la localisation sur une trajectoire de 320 mètres (Figure 1) réalisée dans un environnement naturel relativement riche en obstacles (Figure 2). Les trajectoires des deux SLAMs ont été alignées à la vérité terrain avec l'algorithme Kabsch-Umeyama[9] (Le détail des trajectoires est disponible en annexe : Kitware [I, II, III] et DLIO [IV, V, VI]) :

	APE mean (m)	APE max (m)	RPE mean (m)	RPE max (m)
Kitware SLAM	0.27	0.90	0.017	0.147
DLIO SLAM	0.10	0.36	0.007	0.135

APE : Absolute Pose Error
RPE : Relative Pose Error

FIGURE 1 – Comparaison du Kitware SLAM avec DLIO sur une trajectoire de 320 mètres en environnement géométriquement riche



FIGURE 2 – Environnement d'essais de l'INRAE Aubière

On observe sur la Figure 1 que DLIO donne de meilleurs résultats que le Kitware mais que les deux SLAMs restent suffisamment fiables pour pouvoir naviguer avec.

Voici maintenant un exemple de comparaison de qualité de la localisation sur une trajectoire de 900 mètres (Figure 3) réalisée dans un environnement naturel relativement pauvre en obstacles (Figure 4). Les trajectoires sont alignées à la vérité terrain uniquement au départ :

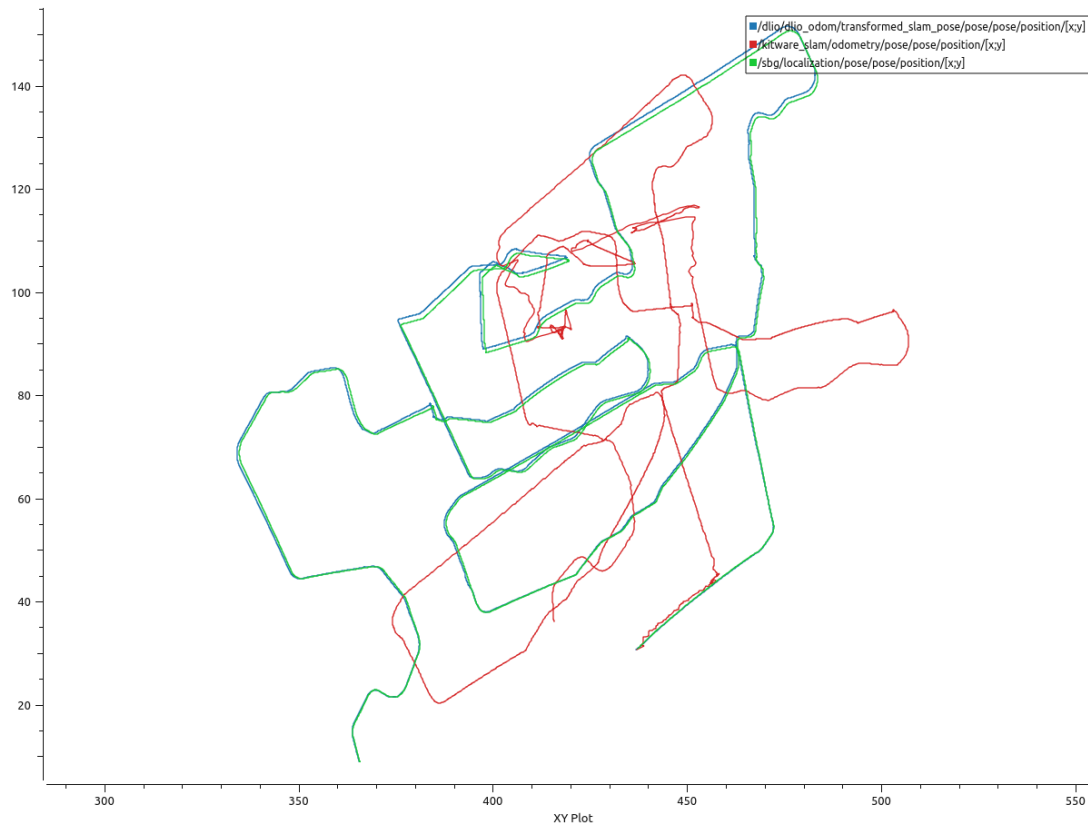


FIGURE 3 – Comparaisons du Kitware SLAM avec DLIO sur une trajectoire de 900 mètres en environnement géométriquement pauvre



FIGURE 4 – Environnement d'essais de l'INRAE Montoldre

On observe sur la [Figure 3](#) que le Kitware SLAM dérive tant qu'il ne suit même plus l'allure de la vérité terrain alors que DLIO reste fiable. L'environnement étant monotone, il devient beaucoup plus difficile pour le Kitware SLAM d'extraire des features permettant de différencier les différents scans LiDAR, faussant complètement les itérations d'ICP et induisant une dérive complète de ce dernier. DLIO se retrouve beaucoup moins impacté par cette pauvreté géométrique de l'environnement car il réalise sa GICP sur l'intégralité de son scan LiDAR en partant d'un à priori fournit par son extrapolation de pose avec l'IMU.

On peut estimer le taux de dérive de DLIO en calculant $\frac{APE(t)}{D(t)}$ ([Figure 5](#)) :

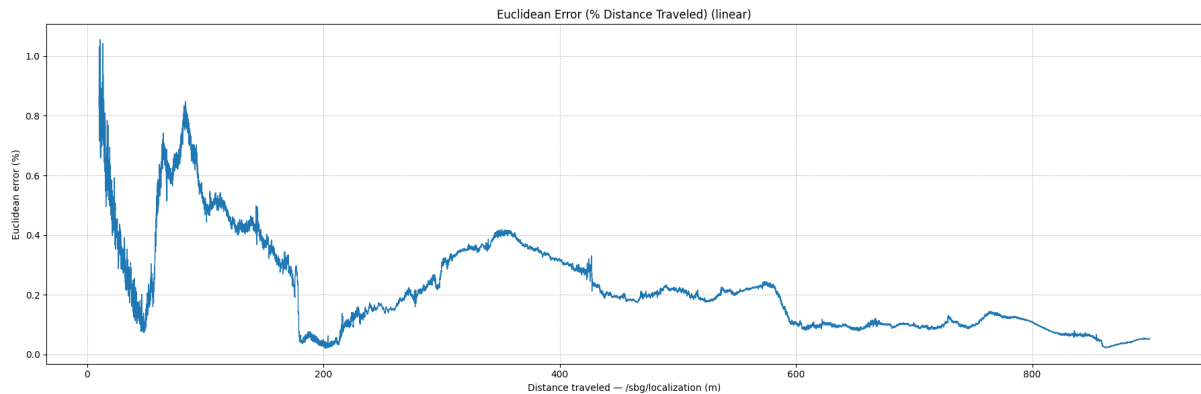


FIGURE 5 – Taux de dérive de DLIO en environnement géométriquement pauvre

Les résultats obtenus Figures 3 et 5 devraient nous permettre de réaliser les scénarios du défi 3 demandant une navigation sans GNSS mais nous obligent tout de même à traiter le problème de discontinuité de la localisation lors du passage SLAM→GNSS.

2 Modifications apportées à DLIO

Pour atteindre les performances illustrées précédemment, des modifications ont été apporté dans le code de DLIO.

Initialement, DLIO donnait une dérive globale plus faible que le Kitware SLAM mais était bruyé, rendant son exploitation dans la stack impossible sans dégrader la carte (Figure 6).

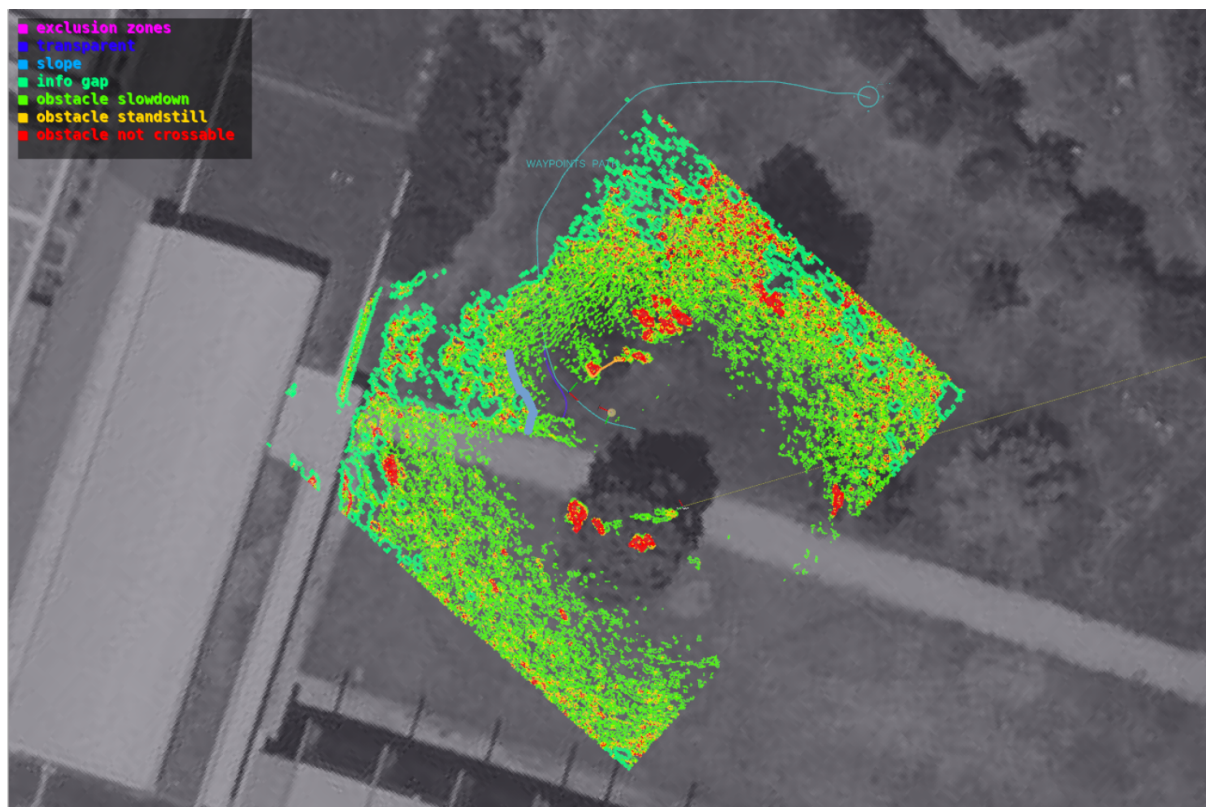


FIGURE 6 – Mapping avec la version originale de DLIO

Après avoir cherché à atténuer ce bruit en modifiant les paramètres de l'observateur géométrique sans grand succès, nous nous sommes rendu compte qu'il y avait un problème conceptuel dans le code du SLAM : **Les fusions d'observations LiDAR et IMU n'étaient pas cohérentes temporellement.**

Notons t_{scan} le timestamp du dernier scan lidar, t_{gicp} le temps de traitement de la GICP pour recalculer le scan courant dans la carte locale et t_{now} le timestamp de l'état courant, correspondant à la dernière fusion IMU.

Dans le code public de DLIO, le temps t_{gicp} est négligé ce qui fait que le résultat de la GICP correspondant à la pose à t_{scan} est fusionné avec un état qui a évolué pendant t_{gicp} pour atteindre $t_{now} \approx t_{scan} + t_{gicp}$. Cette incohérence temporelle fait que l'observateur géométrique calcule une erreur anormalement élevée qu'il cherche à compenser, tirant l'état du robot vers une pose passée et faussant complètement l'estimation des twists et des biais de l'IMU.

Pour corriger ce problème, nous avons ajouté un buffer des états fusionnés calculés par l'observateur et modifié le fonctionnement de l'observateur. Maintenant, les observations LiDAR avec l'état sont fusionnées à t_{scan} en utilisant le buffer d'état pour interpoler l'état à t_{scan} . Comme l'état a continué d'évoluer durant le traitement de la GICP, on vient rétropropager les

observations IMU à l'état à t_{scan} corrigé par la GICP pour remplacer l'état à priori à t_{now} par l'état intégrant la correction GICP.

Cette modification nous a alors permis d'octroyer plus d'importance à l'observation géométrique de la GICP dans l'observateur, rendant les corrections plus agressives pour limiter la dérive globale.

Enfin, une dérive anormale de l'état en sortie de l'observateur comparée aux sorties brutes de la GICP (Figure 7) a été corrigé en utilisant l'IMU de la SBG (40Hz) ou de la Xsens (100Hz) (Figure 8) au lieu de celle du Ouster (100Hz) qui était plus bruitée.

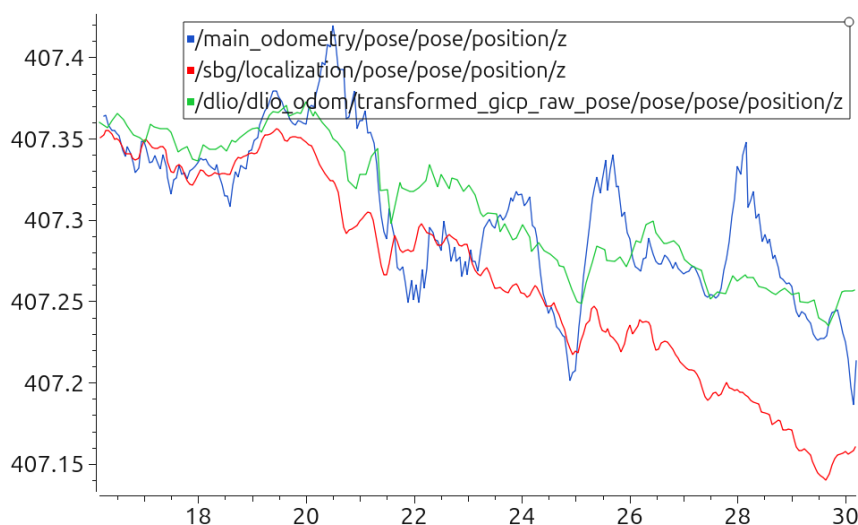


FIGURE 7 – Position en Z de DLIO estimée avec l'IMU du Ouster

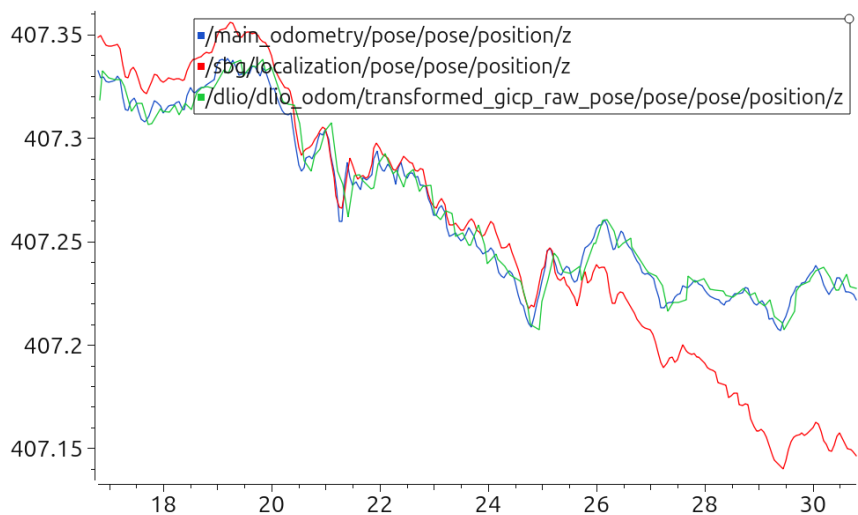


FIGURE 8 – Position en Z de DLIO estimée avec l'IMU de la Xsens

Si on compare le SLAM avant et après modifications sur un même jeu de données, on obtient les résultats [Figure 9](#) :

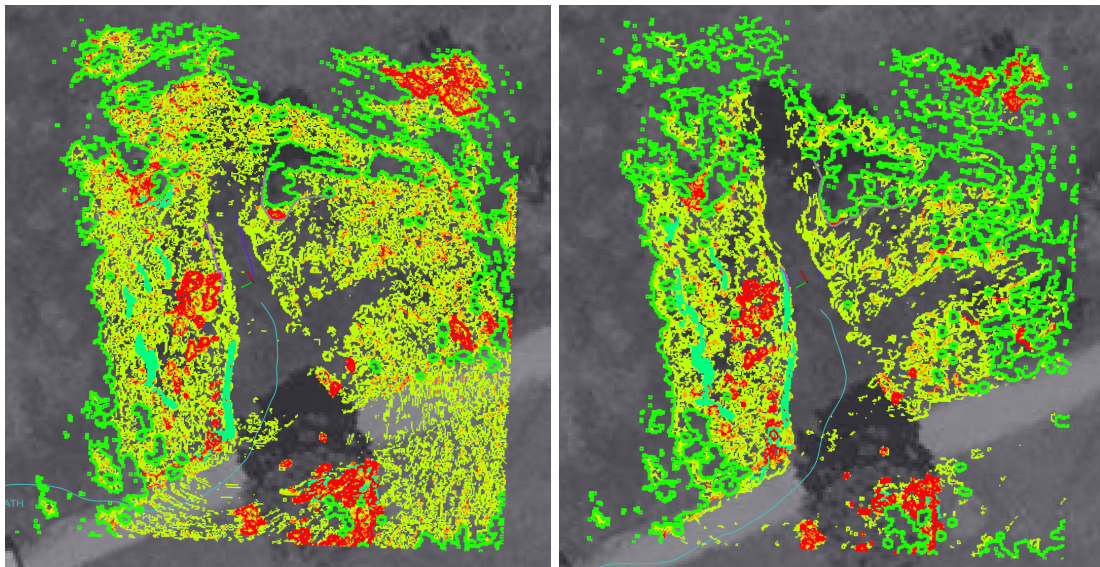


FIGURE 9 – Comparaison des mappings avant et après modification de DLIO

De plus, les résultats [Figure 9](#) ont été réalisés avec un pc suffisamment puissant pour que la GICP soit réalisée en 20ms en moyenne. Sur la Jetson Orin présente sur le robot, ce temps de traitement est d'environ 60ms, ce qui augmentait considérablement le bruit avant les modifications.

3 Modifications de la gestion de la localisation

La substitution du Kitware SLAM par DLIO et la gestion de la discontinuité de la localisation lors du passage SLAM→GNSS ont entraîné diverses modifications dans la brique de localisation et dans celle de mapping.

3.1 Brique de localisation

3.1.1 Intégration de DLIO

Pour intégrer DLIO au reste de la stack, il faut exprimer ses coordonnées dans le repère *map*. Auparavant, la position GNSS était injectée dans le Kitware SLAM et dans l'EKF associé comme pose initiale lors d'un passage en SLAM et le noeud de localisation souscrivait simplement à la sortie de l'EKF.

Cette méthode n'étant pas applicable à DLIO, nous avons modifié le noeud de localisation pour qu'il respecte la norme REP 105.

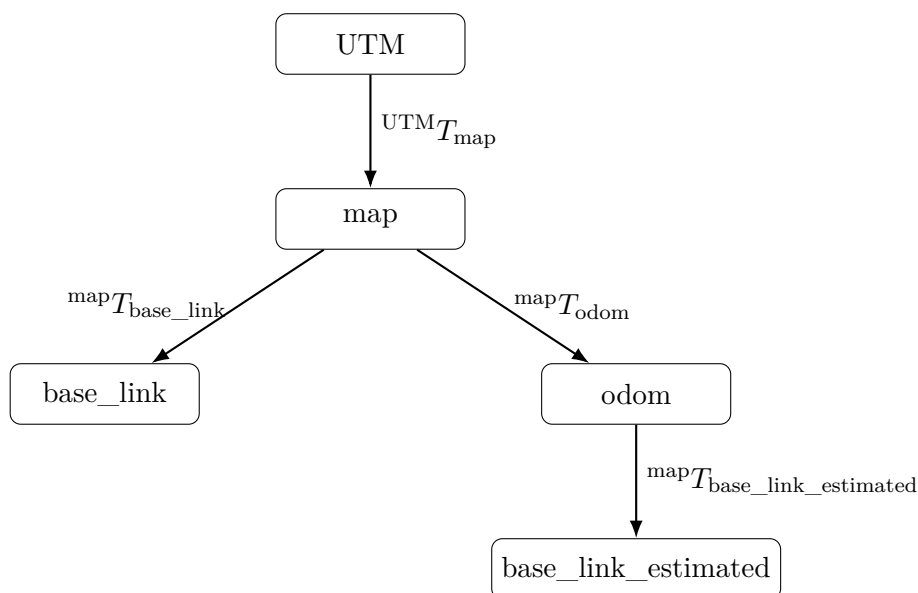


FIGURE 10 – Arbre des transformations TF utilisé pour la localisation

Le SLAM DLIO travaille uniquement dans son repère *odom* dont l'origine est la pose du robot au lancement du SLAM.

Le noeud de localisation gère maintenant la publication de la transformation ${}^{\text{map}}T_{\text{base_link}}$ utilisée par le reste de la stack, le repère *map* étant un repère fixe initialisé selon l'endroit où les tests se déroulent (Aubière, Montoldre, Bourges...).

Lorsque la localisation utilisée est le GNSS, la position du capteur dans le repère UTM est exprimée dans *map* et publiée telle quelle. Lors d'un passage en SLAM, le noeud de localisation vient calculer la transformation ${}^{\text{map}}T_{\text{odom}}$ permettant d'aligner les repères *base_link* et *base_link_estimated* dans *map*. La transformation ${}^{\text{map}}T_{\text{base_link}}$ publiée est alors :

$${}^{\text{map}}T_{\text{base_link}} = {}^{\text{map}}T_{\text{base_link_estimated}} = {}^{\text{map}}T_{\text{odom}} \times {}^{\text{odom}}T_{\text{base_link_estimated}}$$

3.1.2 Ajout d'un modèle de covariance pour le SLAM

3.2 Brique de mapping

3.2.1 Copie locale de la carte avant le saut de localisation

3.2.2 Effacement des mappings avec une localisation incertaine

Références

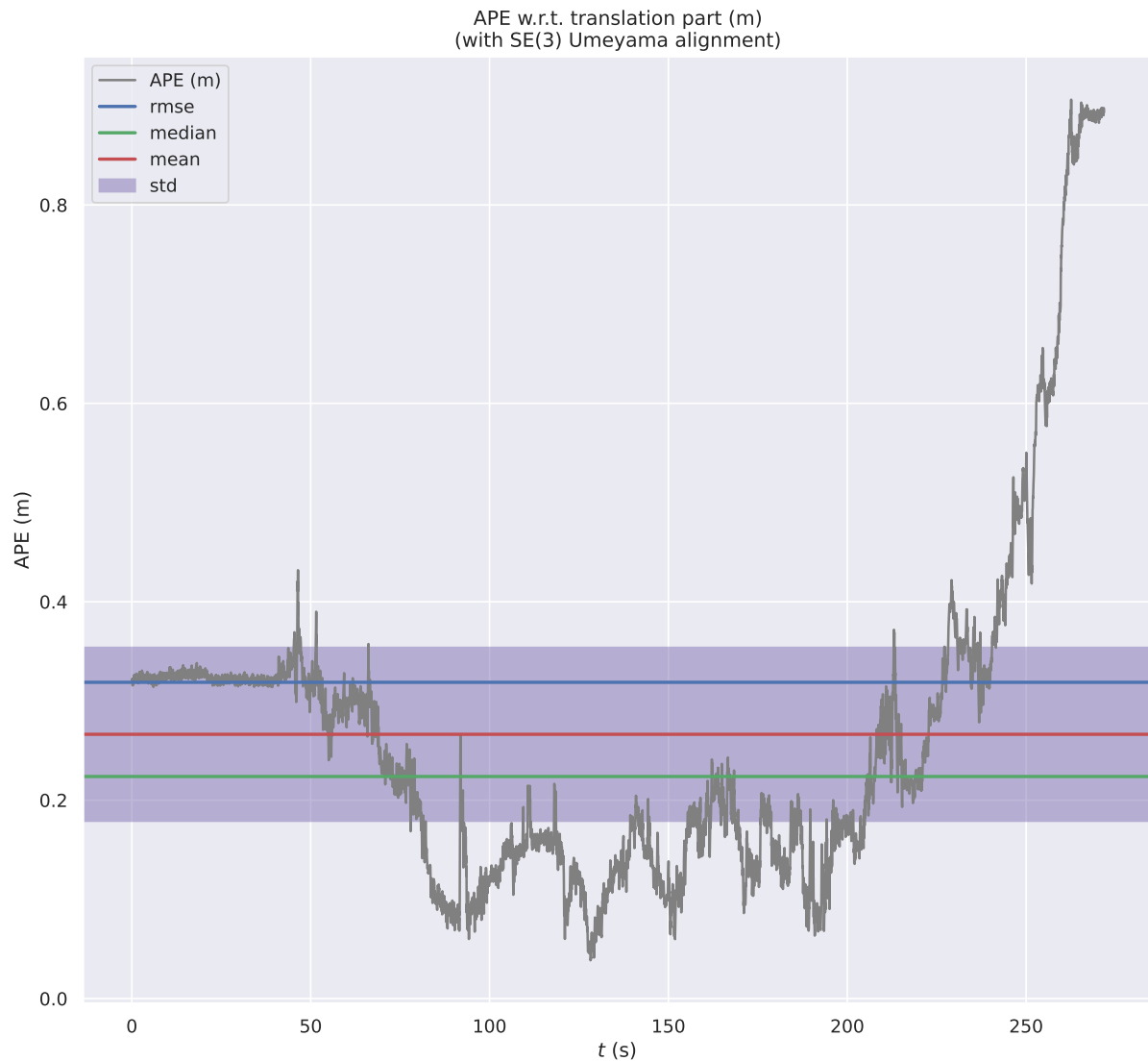
- [1] J. Zhang and S. Singh, “Loam : Lidar odometry and mapping in real-time,” in *Proceedings of Robotics : Science and Systems (RSS)*, 2014.
- [2] Kitware, “Kitware SLAM : Open-source LiDAR processing library.” <https://gitlab.kitware.com/keu-computervision/slam>. Dépôt GitLab officiel.
- [3] Kitware, “Lidar SLAM : Spotlight on Kitware’s open-source library.” <https://www.kitware.com/lidar-slam-spotlight-on-kitwares-open-source-library/>, 2022. Blog officiel.
- [4] K. Chen, R. Nemiroff, and B. T. Lopez, “Direct lidar-inertial odometry : Lightweight lio with continuous-time motion correction,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2023.
- [5] A. Segal, D. Haehnel, and S. Thrun, “Generalized-icp,” in *Proceedings of Robotics : Science and Systems (RSS)*, 2009.
- [6] K. Koide, M. Yokozuka, S. Oishi, and A. Banno, “Voxelized gicp for fast and accurate 3d point cloud registration,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2021.
- [7] W. Xu, Y. Cai, D. He, J. Lin, and F. Zhang, “FAST-LIO2 : Fast direct LiDAR-inertial odometry,” *IEEE Transactions on Robotics*, vol. 38, no. 4, pp. 2053–2073, 2022.
- [8] T. Shan, B. Englot, D. Meyers, W. Wang, C. Ratti, and D. Rus, “LIO-SAM : Tightly-coupled lidar inertial odometry via smoothing and mapping,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 5135–5142, 2020.
- [9] S. Umeyama, “Least-squares estimation of transformation parameters between two point patterns,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 13, no. 4, pp. 376–380, 1991.

Annexes

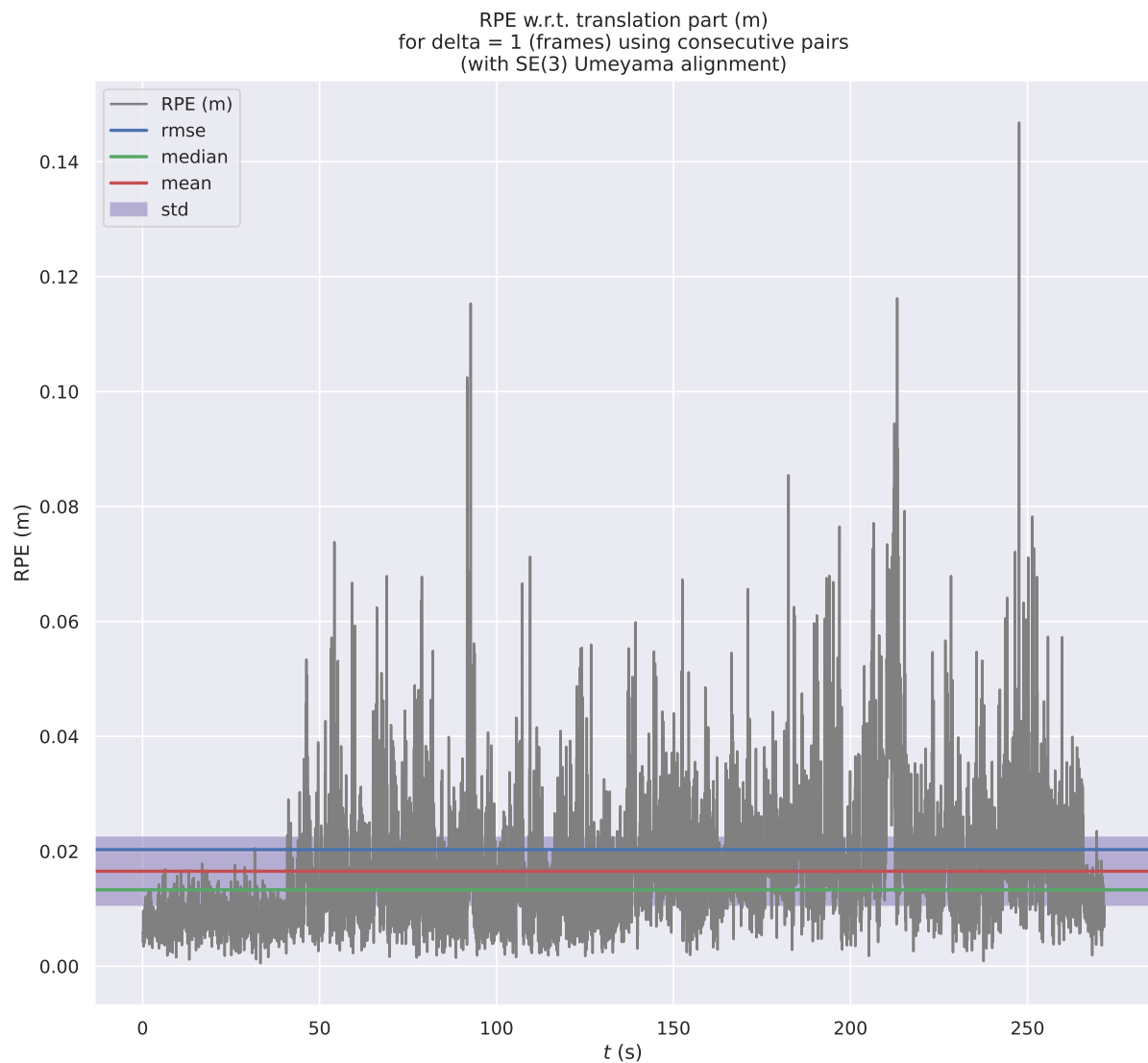
Table des annexes

Annexe I : Absolute Pose Error Kitware SLAM	I
Annexe II : Relative Pose Error Kitware SLAM	II
Annexe III : Trajectoire Kitware SLAM	III
Annexe IV : Absolute Pose Error DLIO	IV
Annexe V : Relative Pose Error DLIO	V
Annexe VI : Trajectoire DLIO	VI

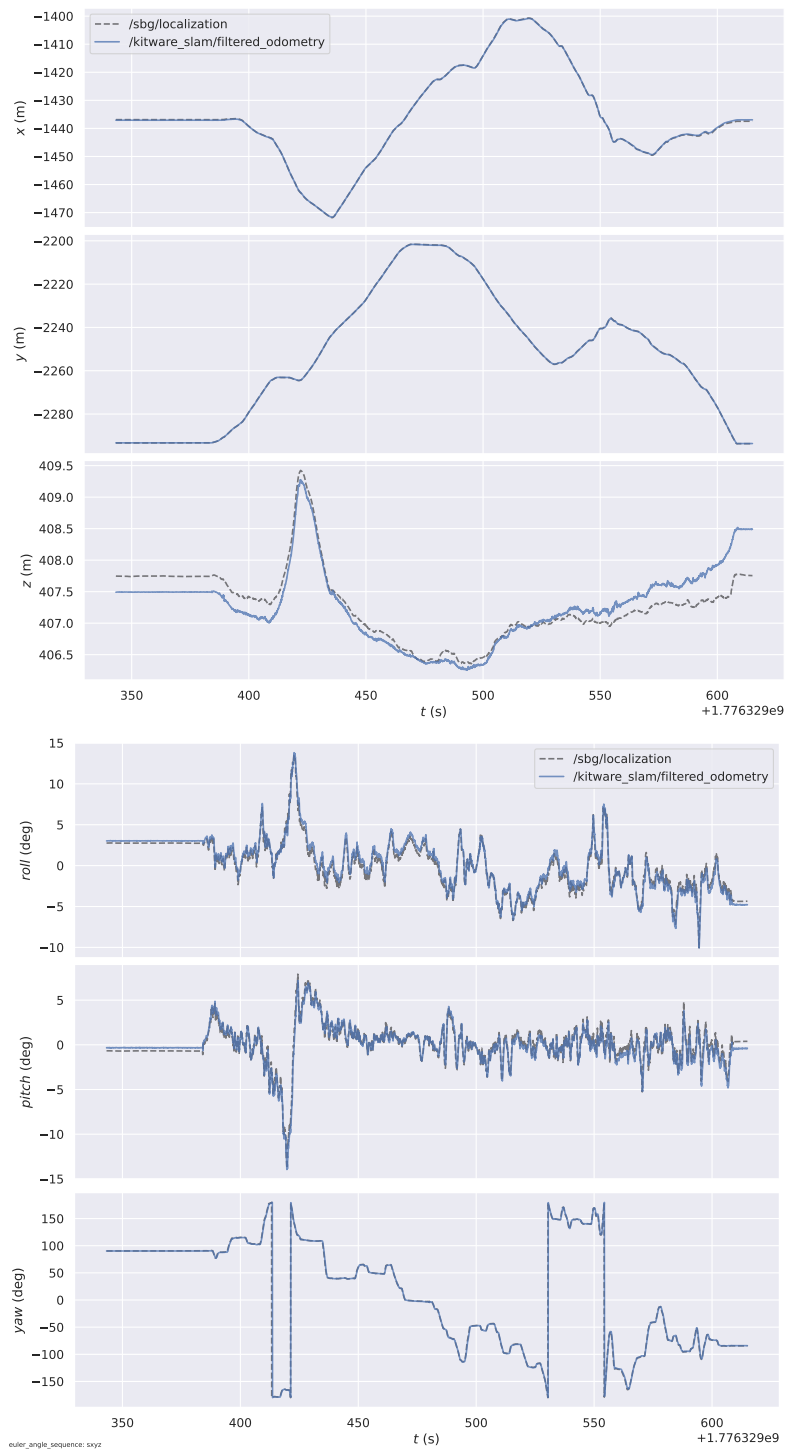
Annexe I: Absolute Pose Error Kitware SLAM



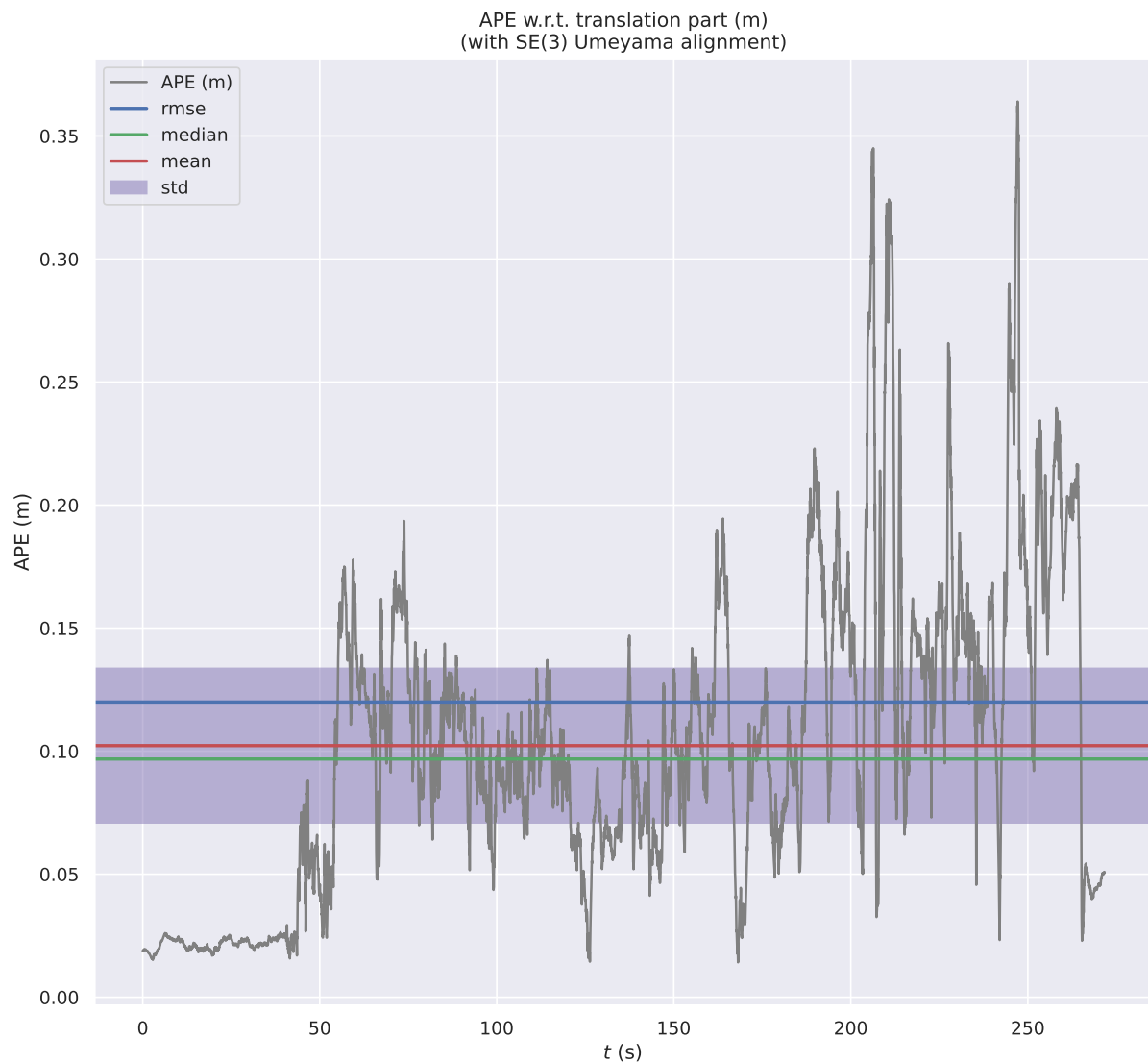
Annexe II: Relative Pose Error Kitware SLAM



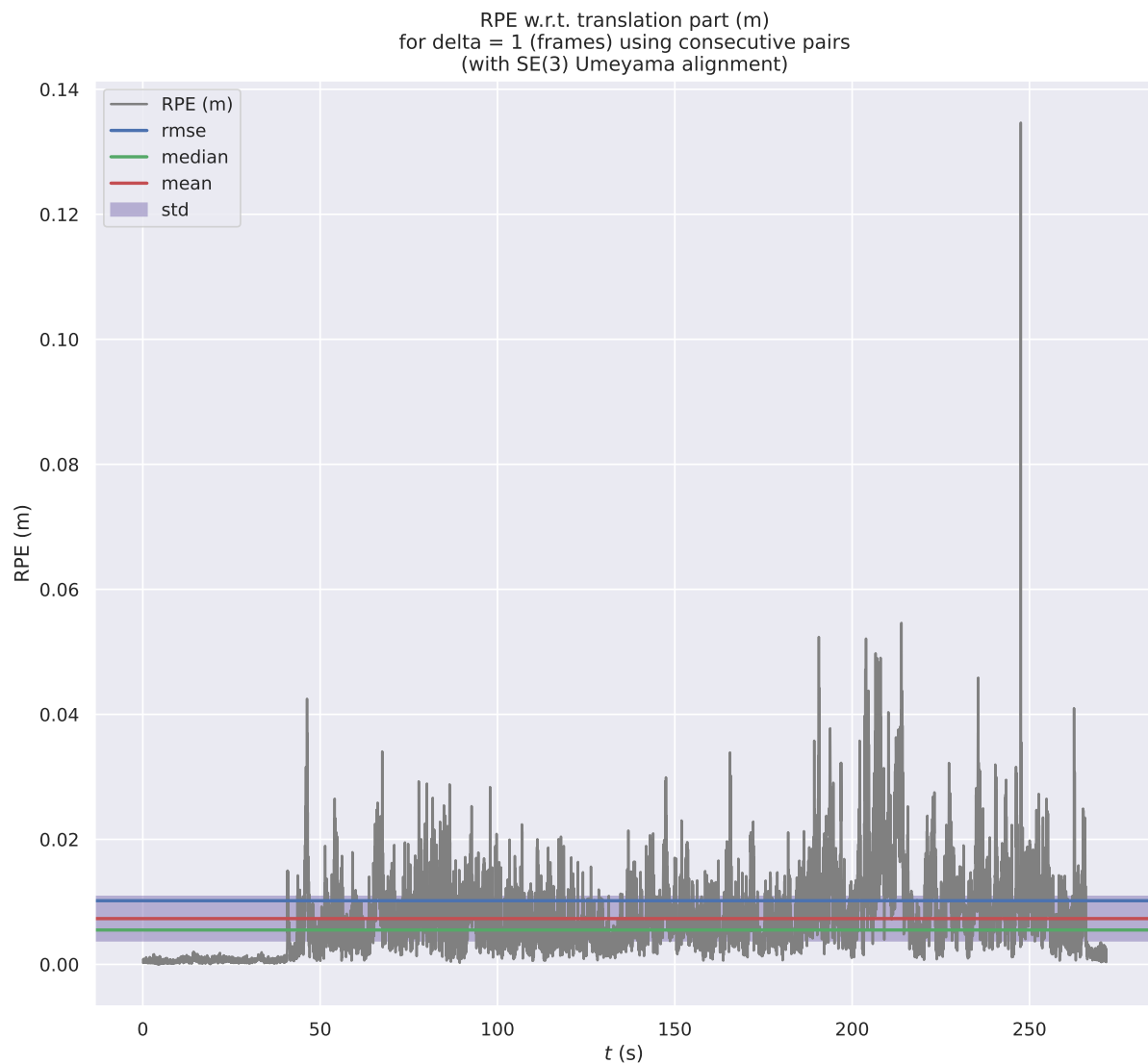
Annexe III: Trajectoire Kitware SLAM



Annexe IV: Absolute Pose Error DLIO



Annexe V: Relative Pose Error DLIO



Annexe VI: Trajectoire DLIO

